UWE Bristol | University of the West of England

Lecture 4: System analysis and design process

Week 4

Majid Mumtaz

Last week recap

- Requirements, Analysis and Modelling
- UML diagrams
- Use case modelling
- Case study – project description, risk analysis, actors, etc.
- Practical – Location Based Shopping App
  - Use case model, use case description

Outline

- System analysis and design review
- UML Class diagrams
  - Representation
  - Identification
  - Relationships
- Case Study
- How to model the way objects interact using Sequence Diagrams

Requirements Elicitation

- The client and developers define the purpose of the system
- The result of this activity is a description of the system in terms of actors and use cases
- Actors represent the external entities that interact with the system
- Actors include roles such as end users, other computers the system needs to deal with (e.g., a central bank computer, a network), and the environment (e.g., a chemical process)

Analysis

- Developers aim to produce a model of the system that is correct, complete, consistent, and unambiguous.
- Developers transform the use cases produced during requirements elicitation into an object model that completely describes the system.
- During this activity, developers discover ambiguities and inconsistencies in the use case model that they resolve with the client.
- The result of analysis is a system model annotated with attributes, operations, and associations.

System Design

- Developers/designers define the design goals of the project and decompose the system into smaller subsystems that can be realized by individual teams.

- Developers also select strategies for building the system, such as the hardware/software platform on which the system will run, the persistent data management strategy, the global control flow, the access control policy, and the handling of boundary conditions.

- The result of system design is a *clear description of each of these strategies, a subsystem decomposition, and a deployment diagram representing the hardware/software mapping of the system*.

System Design cont.

- **The system design** phase specifying the structure of how a software system will be written and function, without actually writing the complete implementation

- Used for describing structure in the use cases

- A transition from "what" the system must do, to "how" the system will do it
  - What classes will we need to implement a system that meets our requirements?
  - What attributes and methods will each class have?
  - How will the classes interact with each other?

Class design

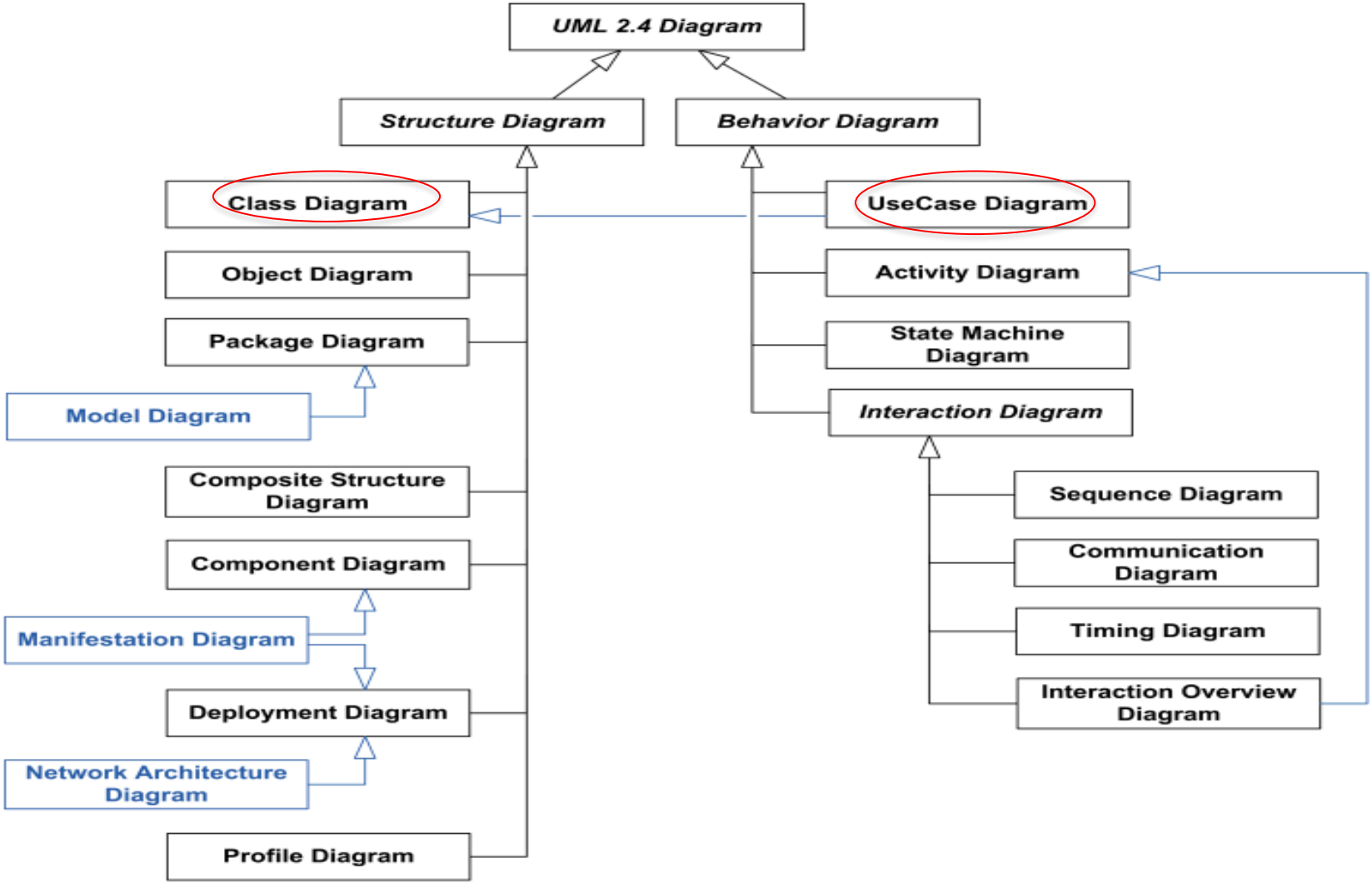
UWE Bristol University of the West of England

- The logical architecture of the system
  - Why is the class included into the architecture?
- Ask the following questions when a class is designed
  - What information or knowledge is the object responsible for maintaining? (described by the attributes of the class)
  - What should the object be able to do? (described by the operations of the class)
  - What other objects does the object need to know to fulfil its responsibilities? (defines associations and aggregations to other classes)
  - Are the responsibilities of objects of this class similar to that of other classes? (defines inheritance relations)

UML class diagrams

- **UML class diagram**: is a diagram used in designing and modelling software to describe classes and their relationships. Class diagrams enable us to model software in a high level of abstraction and without having to look at the source code.

- What are some things that are **not represented in a UML class diagram**?
  - algorithmic details; how a particular behavior is implemented


UML 2.4 Diagram
Structure Diagram
Behavior Diagram
Class Diagram
Object Diagram
Package Diagram
Model Diagram
Composite Structure Diagram
Component Diagram
Manifestation Diagram
Deployment Diagram
Network Architecture Diagram
Profile Diagram
UseCase Diagram
Activity Diagram
State Machine Diagram
Interaction Diagram
Sequence Diagram
Communication Diagram
Timing Diagram
Interaction Overview Diagram

Class Diagram cont.

UWE Bristol University of the West of England

- What makes a class model good?
  - Build a system quickly and cheaply to the satisfaction of the client
  - Build a system that is easy to maintain and adapt

- **Identifying classes**
  - A class describes a set of objects with an equivalent role
  - Identify candidate classes by picking all nouns and noun phrases out of a requirement specification of a system
  - Discard candidates which appear to be inappropriate (redundant, vague, an event or operation, meta-language, outside the scope of the system, an attribute)

What kind of things are classes?

UWE Bristol University of the West of England

- Tangible (real world things): vehicles, furniture, credit/debit cards, insurance policies, buildings, animals, scanners, keyboards etc.
- Roles (played by people or organizations): student, teacher, administrator, patient, clerk, employee, nurse, homeowner, department, region, sales office etc.
- Events: paying bills, an airplane flight, booking tickets
- Interactions: checking out a book at the library, withdrawing money from an ATM, making a deposit through the ATM etc.

- First two are much more common sources for classes – the other two might help to find and name associations between them

Class naming convention

- A class name shall be a noun, or a noun phrase made up of several words
  - All the words shall be initial-capitalized.
  - Use a singular noun for class name
  - Choose a meaningful and self-descriptive class name
  - E.g., **B**ook, **J**ournal, **C**opy (of book), **L**ibraryMember, **M**emberOfStaff
- Be sure to use terminology and names the user of the information system will be comfortable with, otherwise user acceptance of the information system may be difficult to obtain
- Different people in a team may propose different class names: ***Shopper***, ***Buyer***, ***Purchaser***, ***Consumer***, and ***Customer***
  - all five are in fact synonyms. Pick one to eliminate confusion and misunderstanding

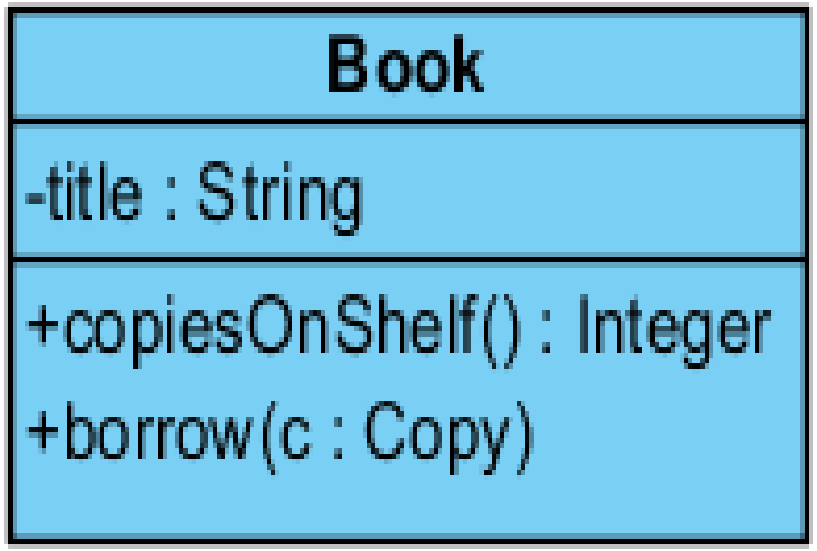
What else do we need in a class?

UWE Bristol University of the West of England

- **Attributes:** state of an object
- **Methods:** captures behaviours
- Define visibility
  - visibility: + public
    # protected
    - private
    ~ package (default)
  - underline <u>static methods</u>
  - parameter types listed as (name: type)
  - omit *return_type* on constructors and when return type is void
- operations / methods

  visibility name (*parameters*) : *return_type*

Class Representation in UML Notation



Class Name (identity)

Class Attribute (state of an object)

Class methods (behaviours)

derived attribute: not stored, but can be computed from other attribute values

Class Diagram cont.

- See examples form Pynative for Python access modifiers to provide encapsulation/information hiding:
- _ for protected
- _ _ for private

Data Hiding using encapsulation

```
class Employee:
   def __init__(self, name, salary, project):
       self.name = name # public - accessible within or outside of a class
       self._project = project # protected - accessible within the class and it's sub classes
       self.__salary = salary # private - accessible only within a class

   def display(self):
       print("Name: ", self.name, 'Salary:', self.__salary, 'project:', self._project)

emp = Employee('Adam', 5000, 'project1')
emp.display()
```

What if there are multiple classes in a class diagram?

- Class Relationships-Connections between classes
- In a class diagram, the connections between classes are shown as arrows. The arrows also show the direction of the connection.

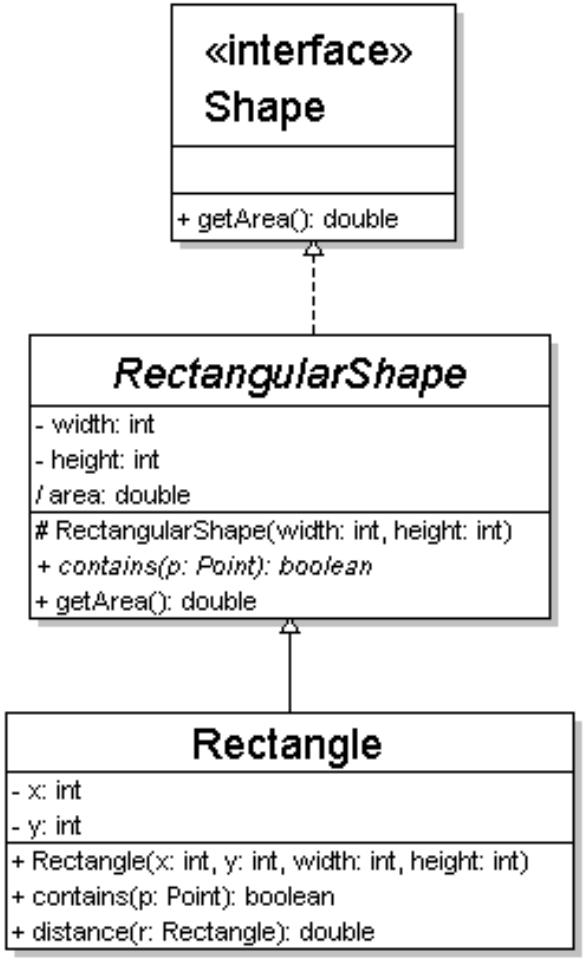
Types of relationships are:

- Generalization
- Association

  - Dependency
  - Aggregation
  - Composition

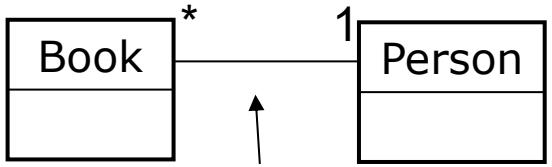
Generalization relationships

- generalization (inheritance) relationships
  - hierarchies drawn top-down with arrows pointing upward to parent
  - line/arrow styles differ, based on whether parent is a(n):
    - <u>class</u>:
      solid line and an arrow
    - <u>abstract class</u>:
      solid line and an arrow
    - <u>interface</u>:
      dashed line and an arrow

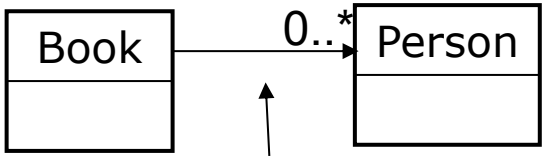


Association

- A relationship between two objects
- Bi-directional or Uni-directional



Bi-directional

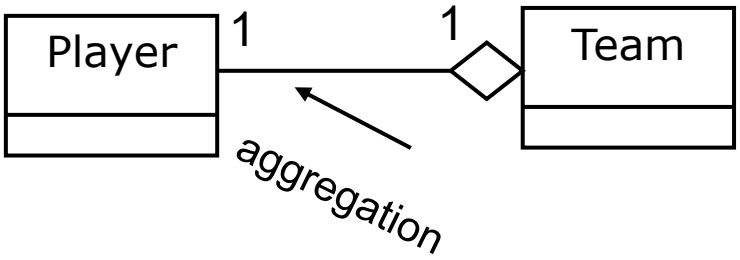


Uni-directional

Association Types

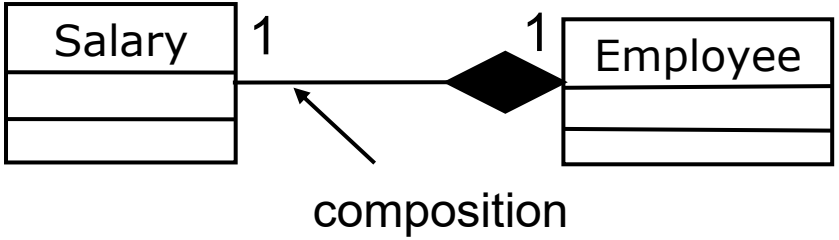
aggregation: "is part of"

- symbolized by a clear white diamond



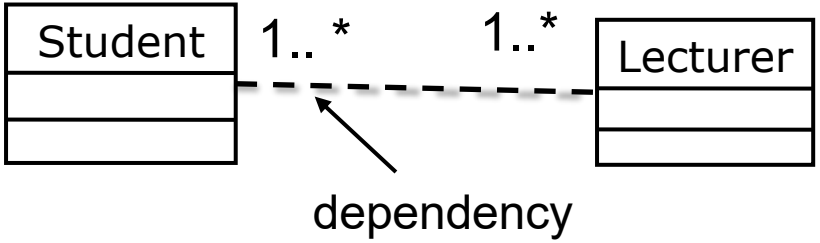
composition: "is entirely made of"

- stronger version of aggregation
- the parts live and die with the whole
- symbolized by a black diamond

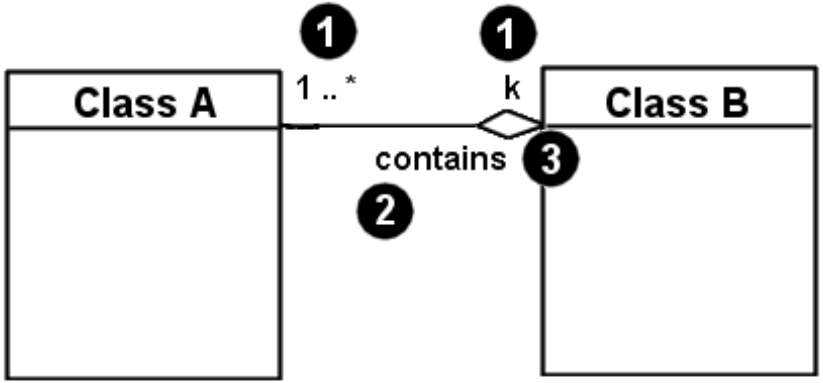


dependency: "uses temporarily"

- symbolized by dotted line
- often is an implementation detail, not an intrinsic part of that object's state



Do we need something else to specify relationships?



1. multiplicity (how many are used)
   - * ⇒ 0, 1, or more
   - 1 ⇒ 1 exactly
   - 2..4 ⇒ between 2 and 4, inclusive
   - 3..* ⇒ 3 or more
2. name (what relationship the objects have)
3. navigability(direction)

Mentimeter

https://www.menti.com/alwbqz2bz3nz

How can we capture/show Class relationships in a code?

Generalization relationships

UWE Bristol University of the West of England

```
Class Parent:
    def func1(self):
        print('This is parent class.func1()')

Class Child(Parent):
    def func2(self):
        print('This is child class.func2()')

Obj = Child()
Obj.func1()
Obj.func2()
```

Inheritance

What will be the output?

For OO programming in Python – A beginner's guide to Python 3 Programming by John Hunt. Chapter 17 onwards on OOP…. Chapter 20 - Class Inheritance

Generalization relationship

```
import abc
class Shape(metaclass=abc.ABCMeta):
      @abc.abstractmethod
      def getArea(self):
            pass

class Rectangle(Shape):
      def __init__(self, x,y):
            self.width = x
            self.height = y

      def getArea(self):
            return self.width*self.height

r = Rectangle(30,20)
print ('area: ',r.getArea())
```

Abstract class

Abstract method

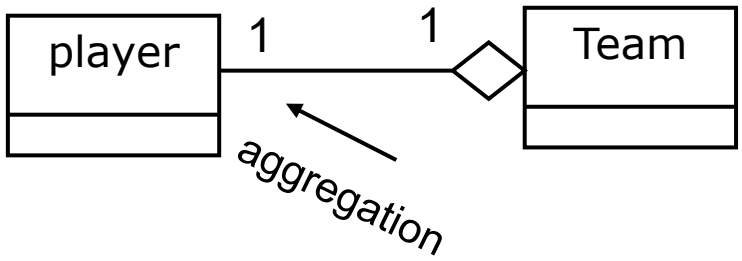
Implementation

Abstract method implementation

More on Abstraction in OOP and Abstraction in Python here

Aggregation

- **aggregation**: "is part of" (also Has-A)
  - symbolized by a clear white diamond

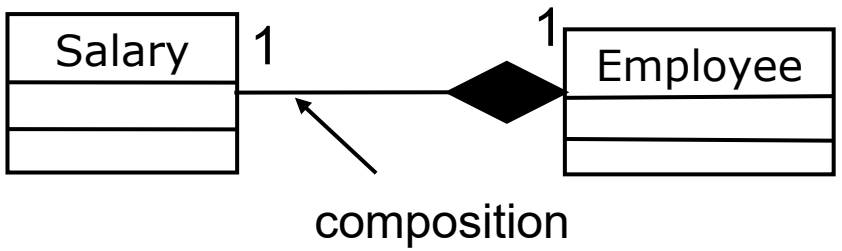



```
class Player:
        def __init__(self, name):
                self.name = name
        def play_match(self):
                print(f"Player {self.name} is playing the match!")

class Team:
        def __init__(self, players): # Team receives existing Player objects
          self.players = players   # Aggregation: Team "has" players
        def start_match(self):
          print("Team is starting the match with its players:")
         for player in self.players:
           player.play_match()
```

Composition

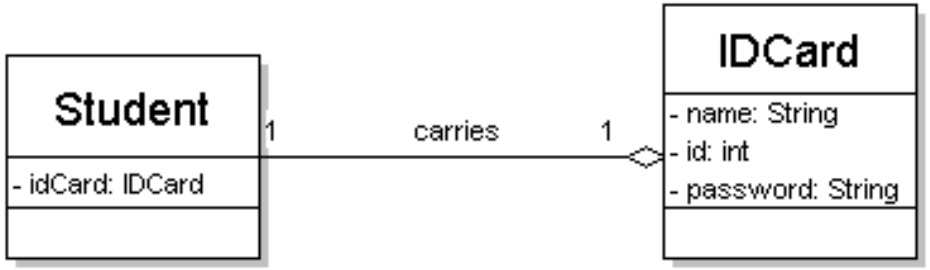
- **composition**: "is entirely made of"

```
 class Salary:
    def __init__(self, pay):
        self.pay = pay

    def get_total(self):
        return (self.pay*12)

class Employee:
    def __init__(self, pay, bonus):
        self.pay = pay
        self.bonus = bonus
        self.obj_salary = Salary(self.pay)

    def annual_salary(self):
        return "Total: " + str(self.obj_salary.get_total() + self.bonus)

obj_emp = Employee(600, 500)
print(obj_emp.annual_salary())
```



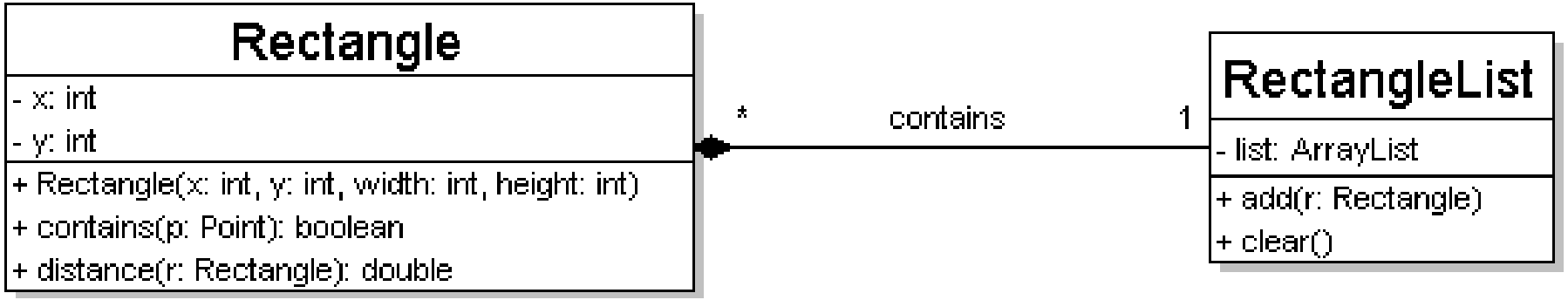
Aggregation and Composition Examples

- one-to-one
  - each student must carry exactly one ID card



- one-to-many
  - one rectangle list can contain many rectangles

Is Diamond direction correct?



Dependency

- **dependency**: "uses temporarily"

```
class Student:
        def __init__(self, name):
                self.name = name
        def get_name(self):
                return self.name
# Pure dependency: Student temporarily
"uses" Printer
        def print_id_card(self, printer):
                printer.print(f"Student ID
Card\nName: {self.name}\nValid: 2025-
2026")
```

```
class Printer:
            def print(self, text):
                print("Printer is printing...")
                print(text)
                print("--- Printed ---\n")
```

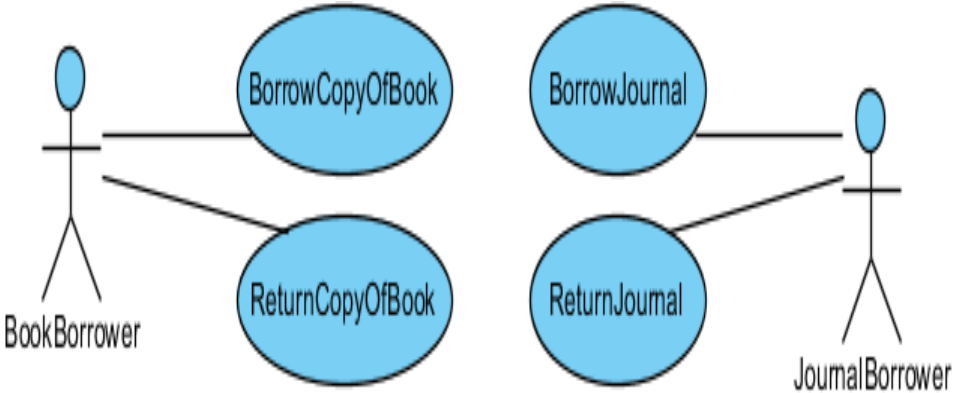
Dependency cont.

- **dependency**: "uses temporarily"

```
# ----------------------------
# Usage - shows pure dependency
# ----------------------------
student = Student("Alex")

# Any printer can be used - no permanent  relationship
library_printer = Printer()
lab_printer = Printer()

# Student depends on Printer only at the moment of calling
student.print_id_card(library_printer)    # ← Dependency in action
student.print_id_card(lab_printer)        # ← Same student, different printer

# Printer can exist and be used without any
Studentstandalone_printer = Printer()
standalone_printer.print("Just a test page from admin")
```

A Case Study

- Development of a computer system for the university library
  - We use an iterative process
  - After discussing priorities with the university, we decided that the first iteration of the system should provide the use cases:
    - Borrow copy of book
    - Return copy of book
    - Borrow journal
    - Return journal



Case Study Cont.

UWE Bristol University of the West of England

- Books and journals:

  The library contains books and journals. It may have several copies of a given book. Some of the books are for short term loans only. All other books may be borrowed by any library member for three weeks. Members of the library can normally borrow up to six items at a time, but members of staff may borrow up to 12 items at one time. Only members of staff may borrow journals.

- Borrowing

  The system must keep track of when books and journals are borrowed and returned, enforcing the rules described above

Identifying classes

- Simplest and best: look for nouns and noun phrases in the system description!

- Then abandon things which are:
  - redundant, outside scope, vague, attributes, operations
    - may need to add some back later

- Similarly, can use verb phrases to identify operations and/or associations

Identifying classes cont.

- Books and journals:

  The library contains books and journals. It may have several copies of a given book. Some of the books are for short term loans only. All other books may be borrowed by any library member for three weeks. Members of the library can normally borrow up to six items at a time, but members of staff may borrow up to 12 items at one time. Only members of staff may borrow journals.

  Borrowing:

  The system must keep track of when books and journals are borrowed and returned, enforcing the rules described above

Identifying classes cont. [Eliminate]

- Books and journals:

  The library contains books and journals. It may have several copies of a given book. Some of the books are for short term loans only. All other books may be borrowed by any library member for three weeks. Members of the library can normally borrow up to six items at a time, but members of staff may borrow up to 12 items at one time. Only members of staff may borrow journals.

  Borrowing:

  The system must keep track of when books and journals are borrowed and returned, enforcing the rules described above

Identifying classes cont. [Eliminate]

library, because it is outside of the scope of our system

short term loan, because a loan is really an action, the lending of a book to a user

member of the library, which is same as library member

week, because it's a measure of time, not a thing

time, because it's outside the scope of the system

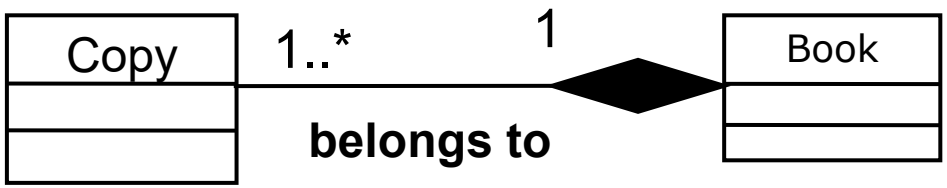
system, because it's part of the meta-language of requirements description

rule, for the same reason

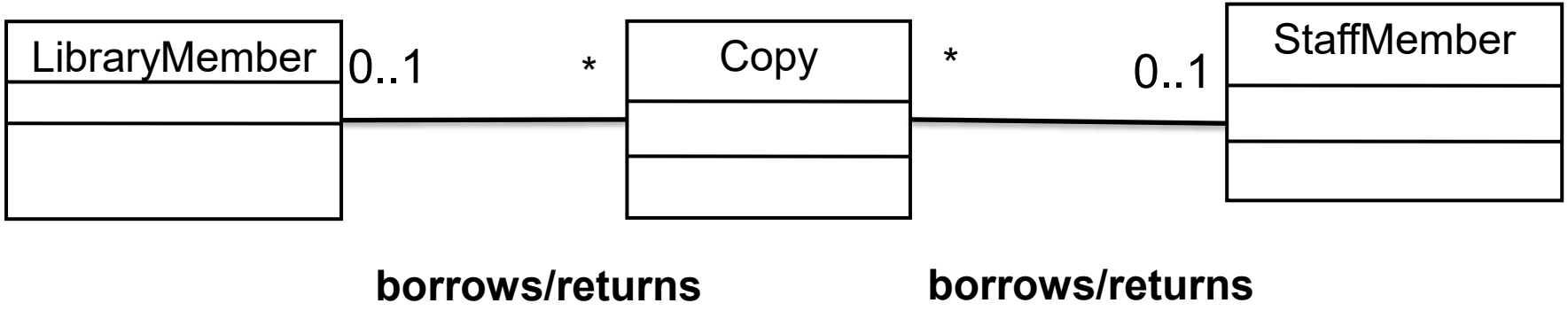
Identifying classes cont.

Left with: book, journal, copy (of book), library member, member of staff.

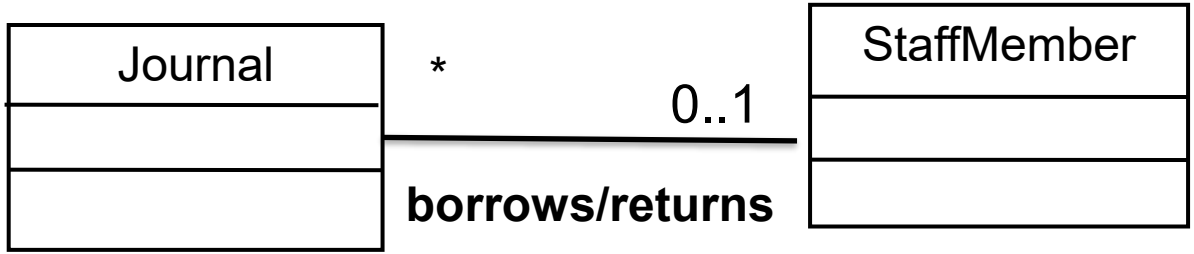
Relationships and multiplicity



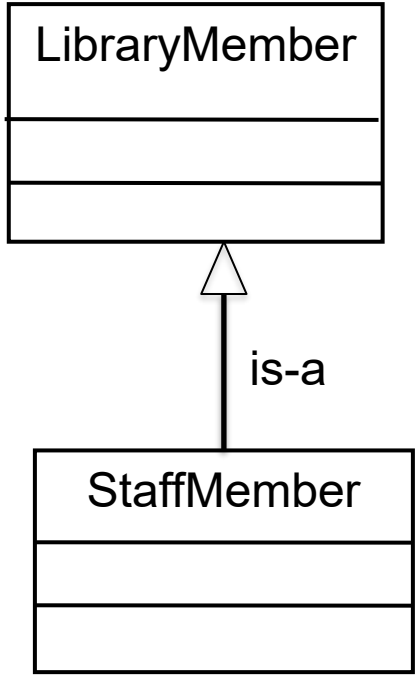
Relationships and multiplicity cont.



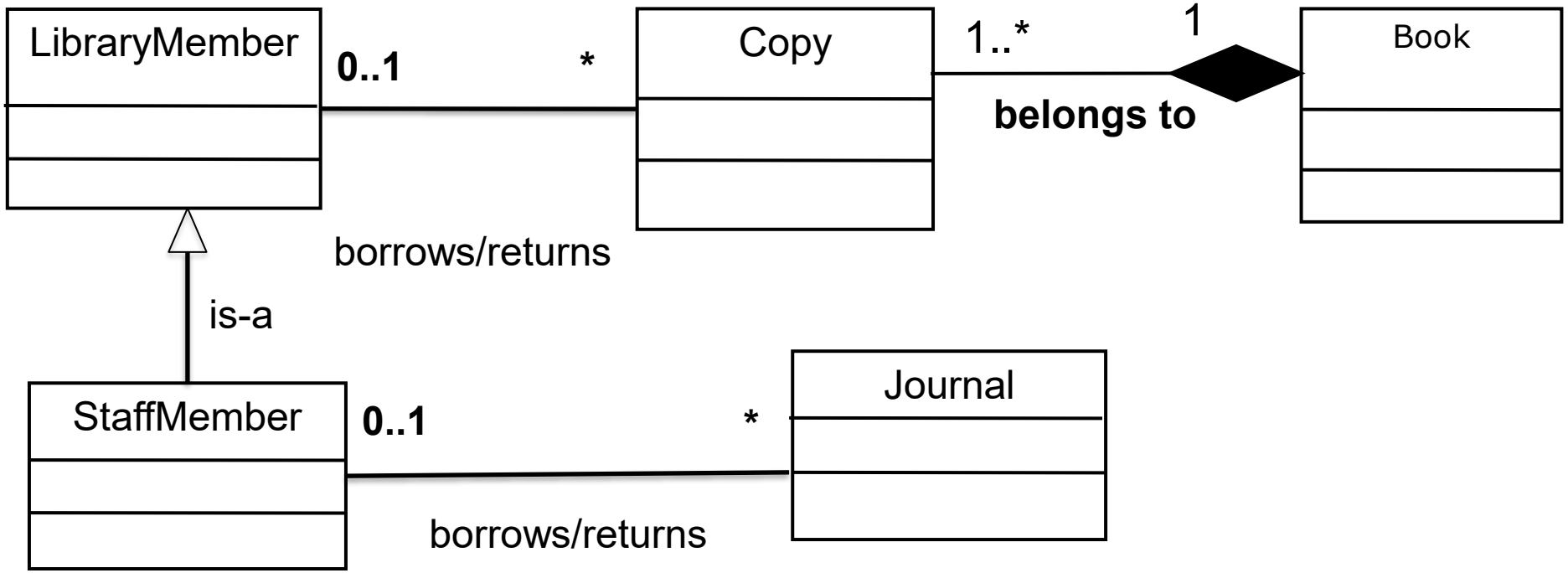
Relationships and multiplicity cont.



Relationships and multiplicity cont.

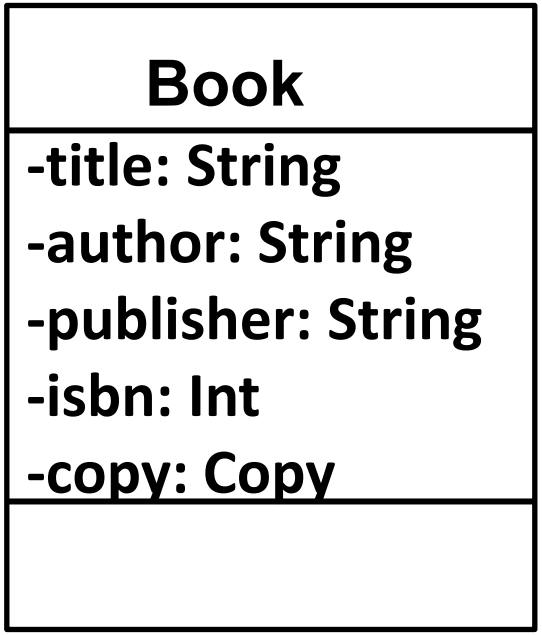


Relationships and multiplicity cont.



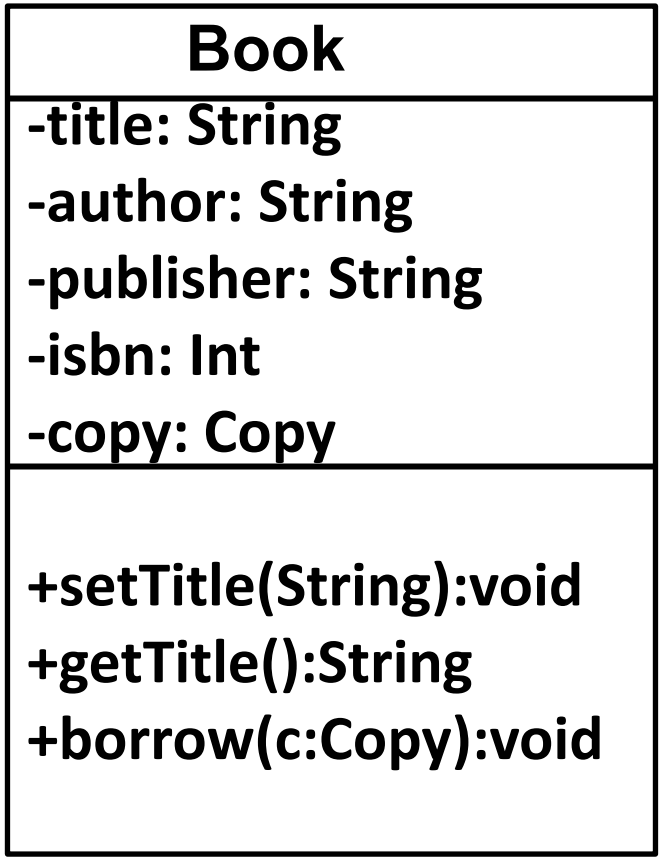
Class Attributes

- Describe data contained in an object of a class
- Don't show attributes that simply implement associations – Eg no 'copies' attribute in the 'Book' class
- Some nouns may become attributes of classes rather than classes themselves



Behaviour/Responsibilities

- Think about what behaviours or responsibilities each class should have
  - these will become methods in our classes
- The use case analysis provides a list of tasks (along with the steps involved from the users perspective) that the software should perform
  - think about which class(es) should be responsible for preforming the tasks necessary for each use case

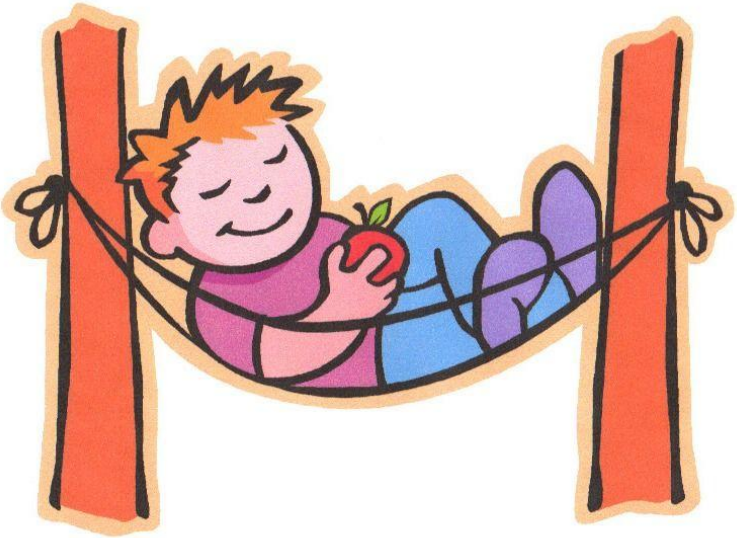


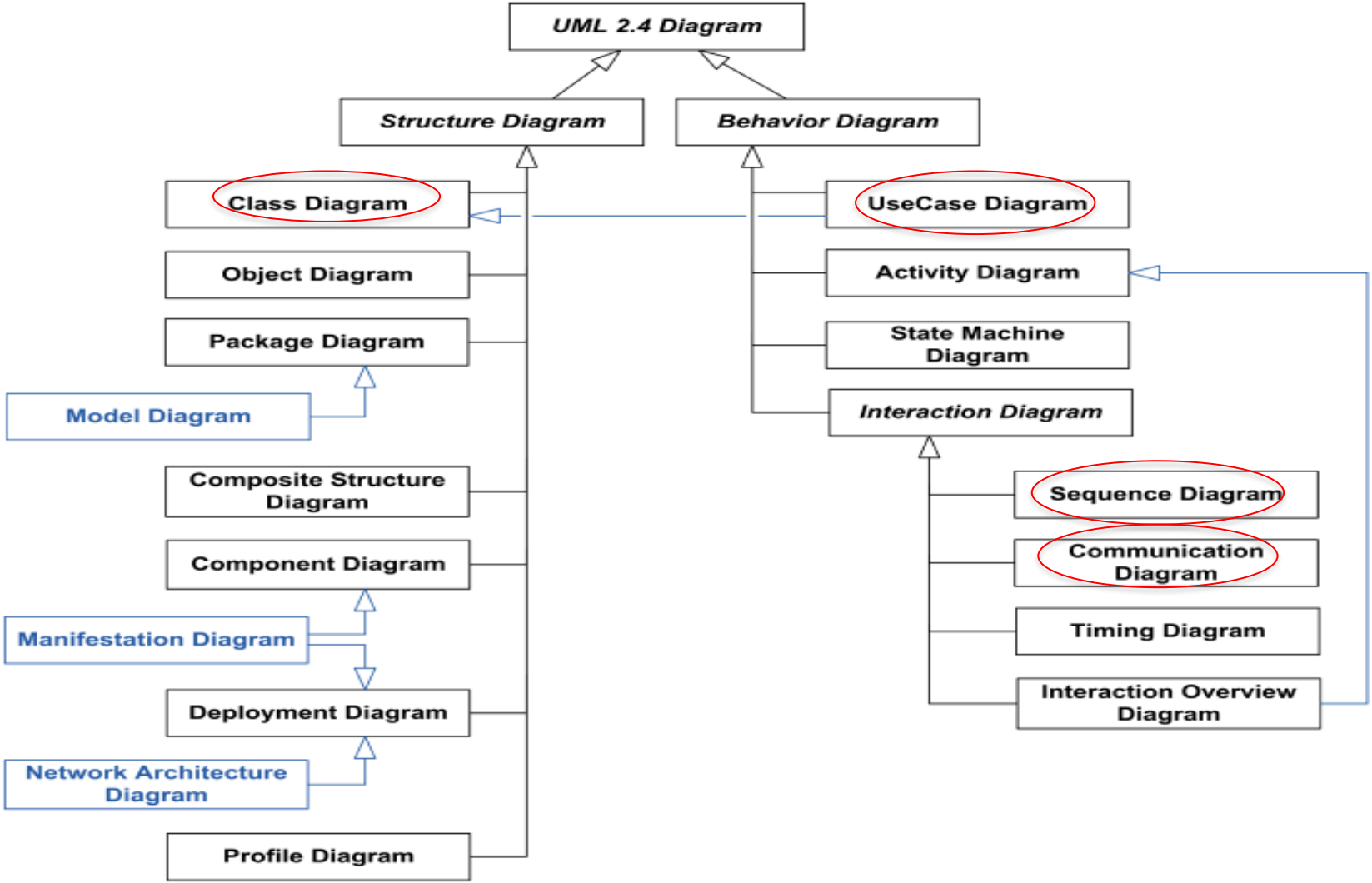
Class Diagram-Summary

- Ensure that the classes are both necessary and sufficient to solve the underlying problem
  - no missing attributes or methods in each class
  - no extra or unused attributes or methods in each class
  - no missing or extra classes

UWE Bristol University of the West of England

Take a Break





University of the West of England

Interaction Diagrams

- They model the behavior of use cases by describing the way groups of objects and entities interact to complete the task of the use case.
- They help us to visualize the interactive behaviour of a system.
- They represent how one or more objects in the system connect and communicate with each other.
- They are used to observe the dynamic behaviour of a system.

Interaction Diagrams cont.

- **Sequence diagrams**
  demonstrate the sequence of events that occur.

- **Communication diagrams**
  demonstrate how objects are statically connected.

- Both diagrams are relatively simple to draw and contain similar elements.

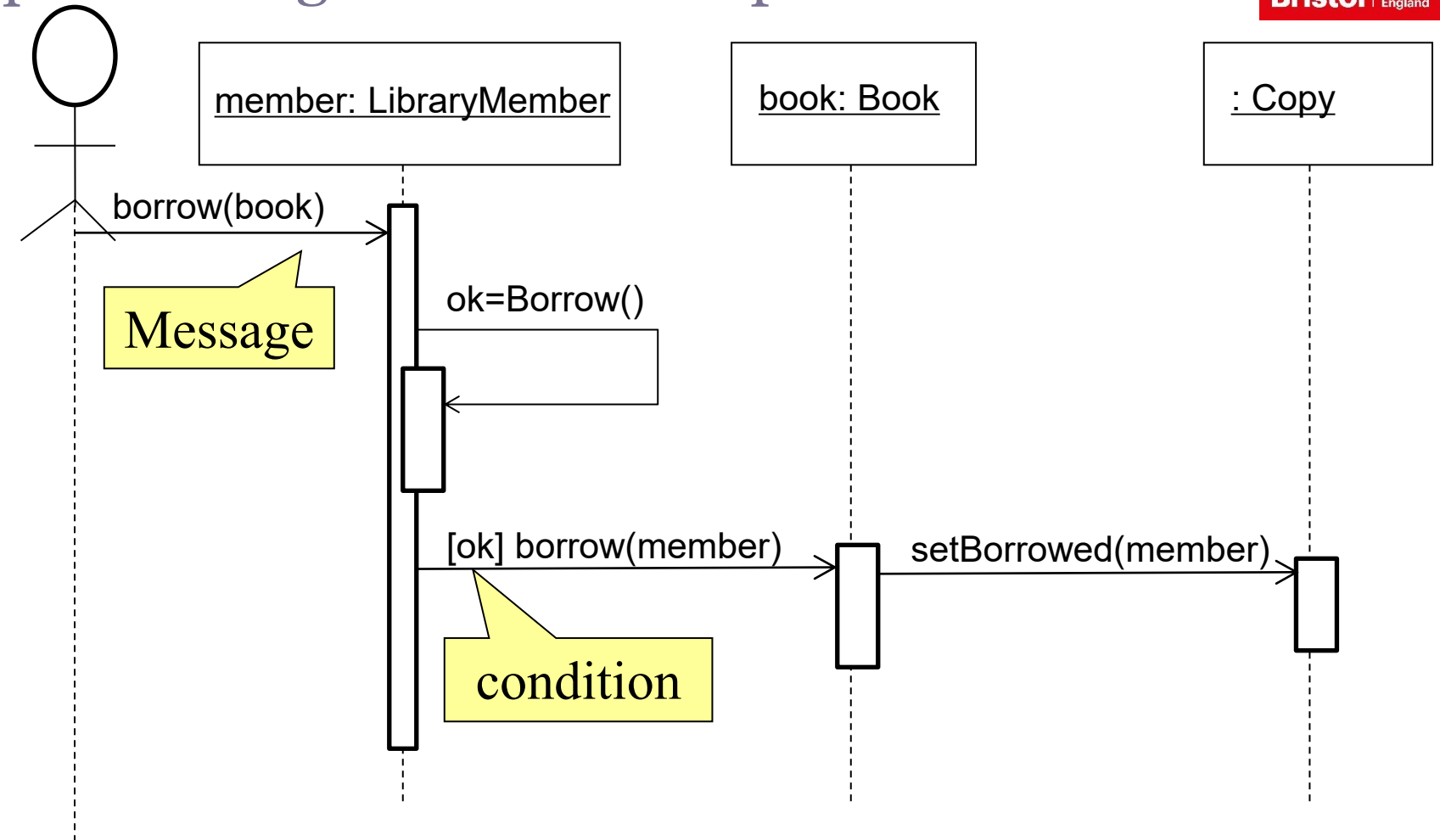
Sequence Vs. Communication Diagrams

- They are essentially semantically equivalent
- Both are used to model the **dynamic aspects of a system in terms of objects interacting by exchanging messages**
- They differ in how the information is presented
- Sequence diagrams emphasize the time ordering of messages, whereas the other one depicts more of an organizational structure

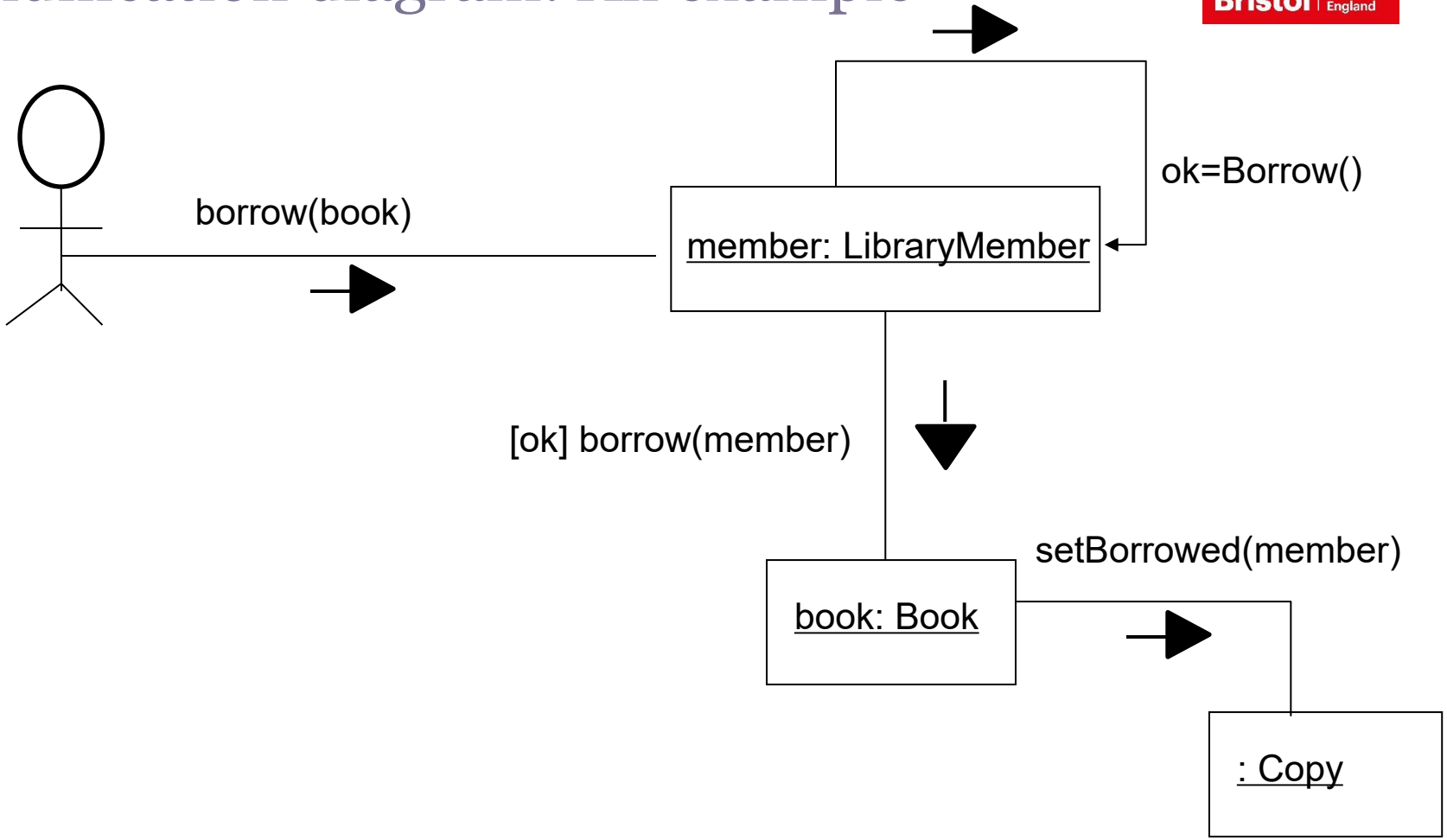
Sequence diagrams (as well as communication diagrams)

- The purpose of UML sequence diagrams
  - to show **how objects interact with one another**
  - to **capture the dynamic behaviour** of a system
  - to **describe the message flow** in the system
  - to **compose the run-time entities** : objects (no classes at run-time)

Sequence Diagrams: An example



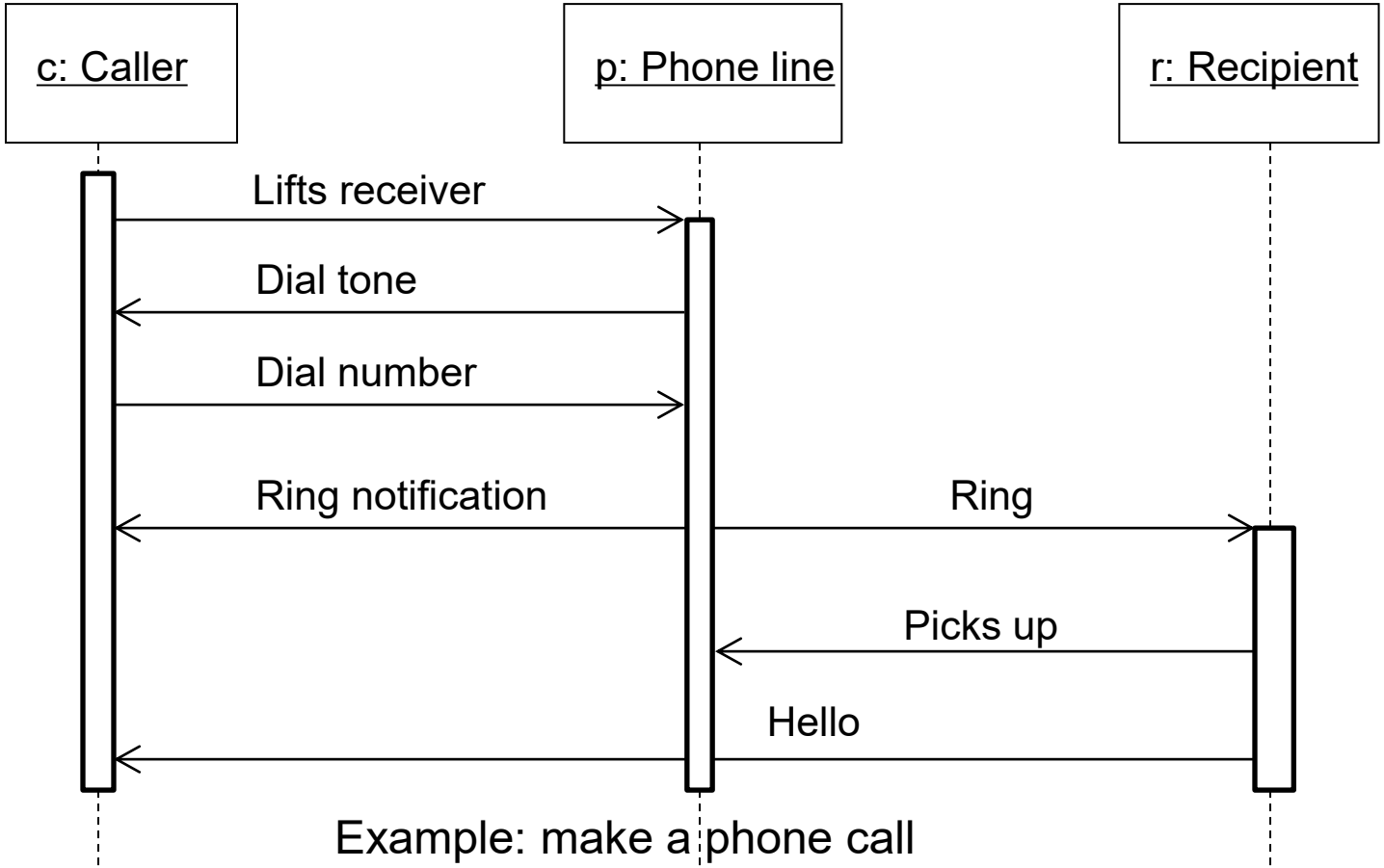
Communication diagram: An example



Sequence Diagrams cont.

- **Participant**: object or entity that acts in the diagram
- **Message**: communication between participant objects
- The **axes** in a sequence diagram:
  - **horizontal**: which object/participant is acting
  - **vertical**: time (down -> forward in time)

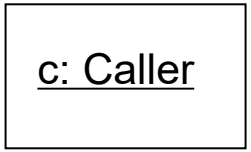
Sequence Diagrams cont.

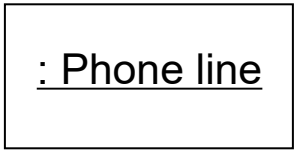


Example: make a phone call

Sequence Diagrams cont.

- Representing objects



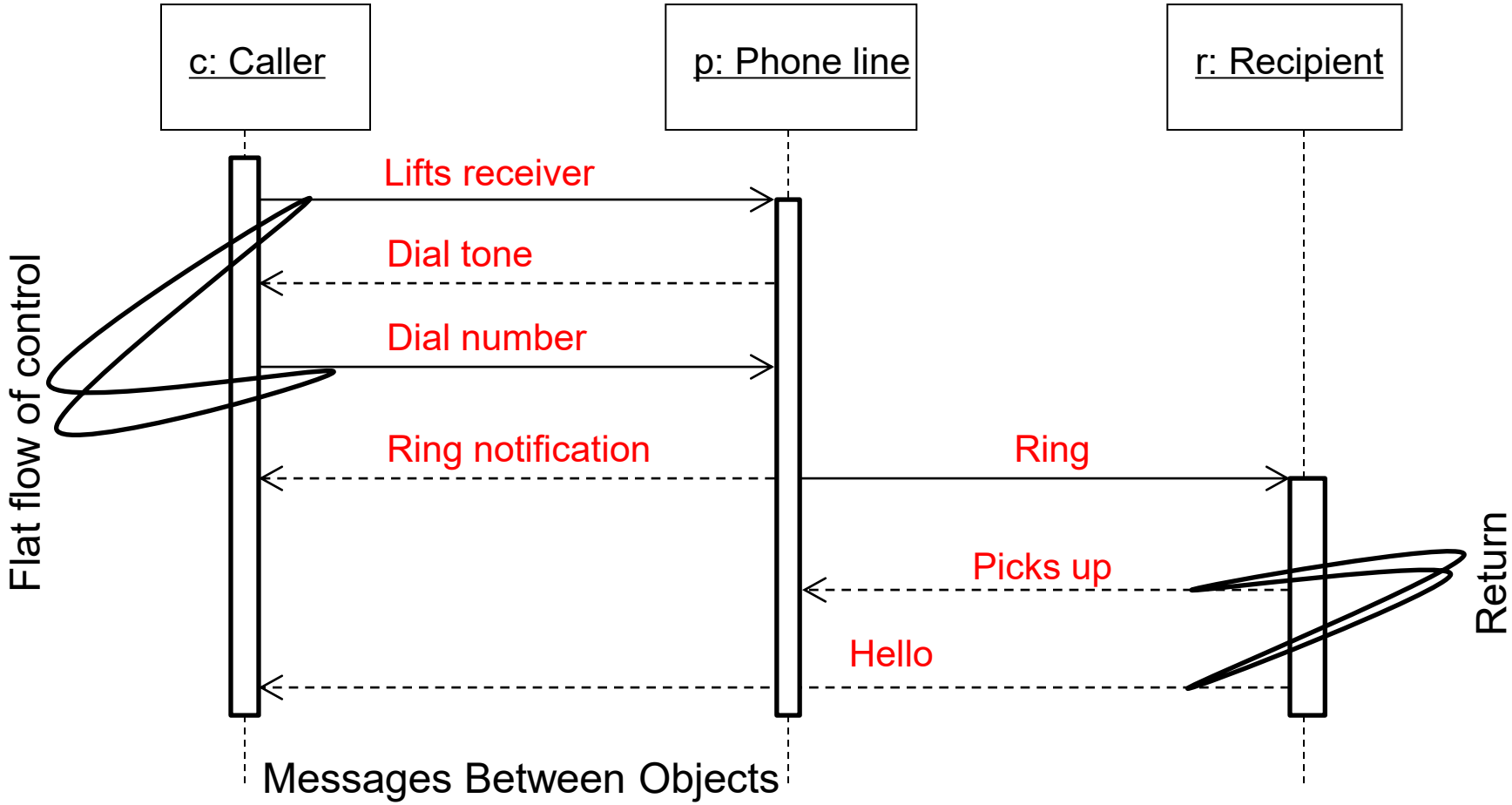


Specific object

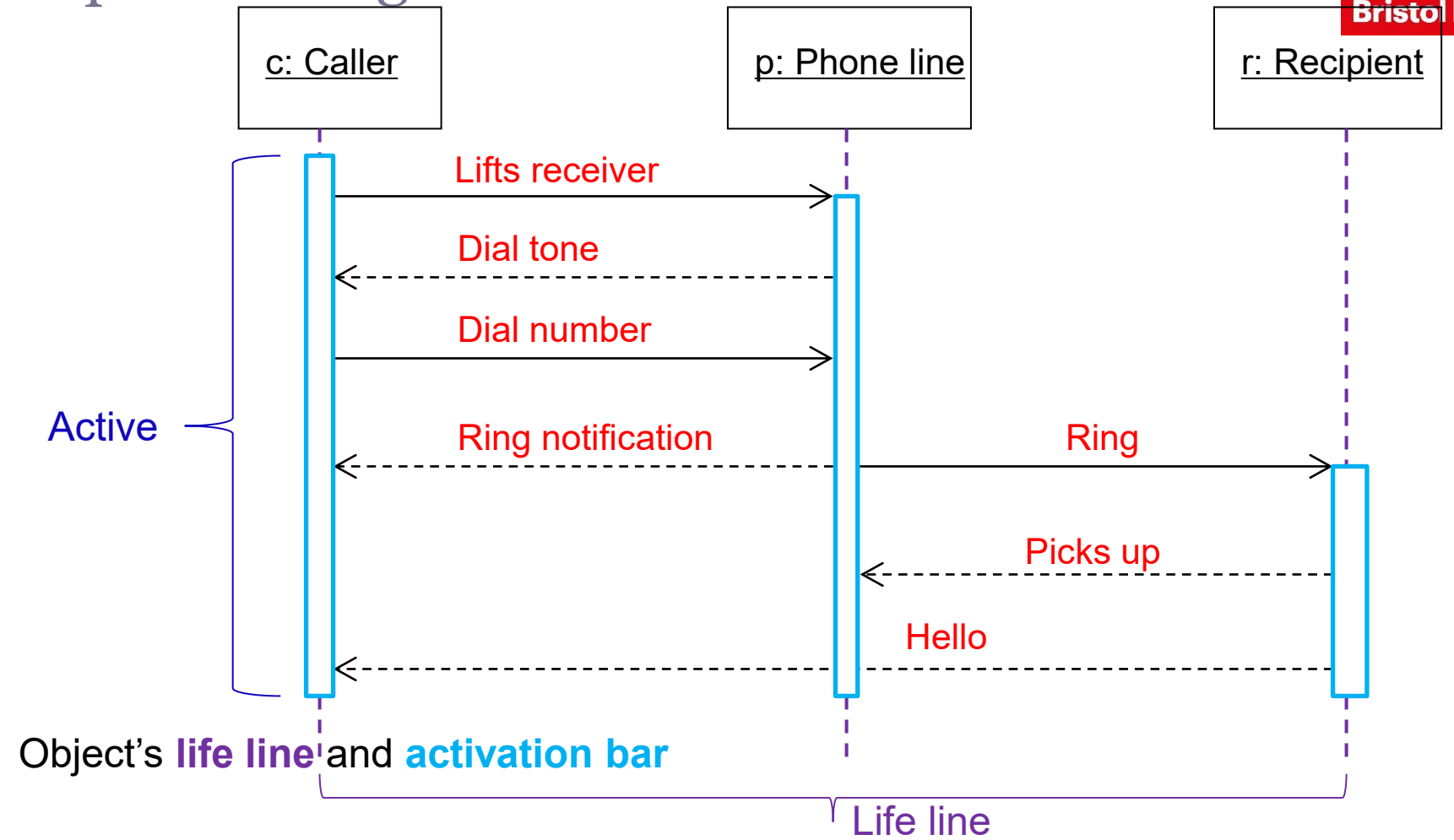
Anonymous object

Unknown object

Sequence Diagrams cont.

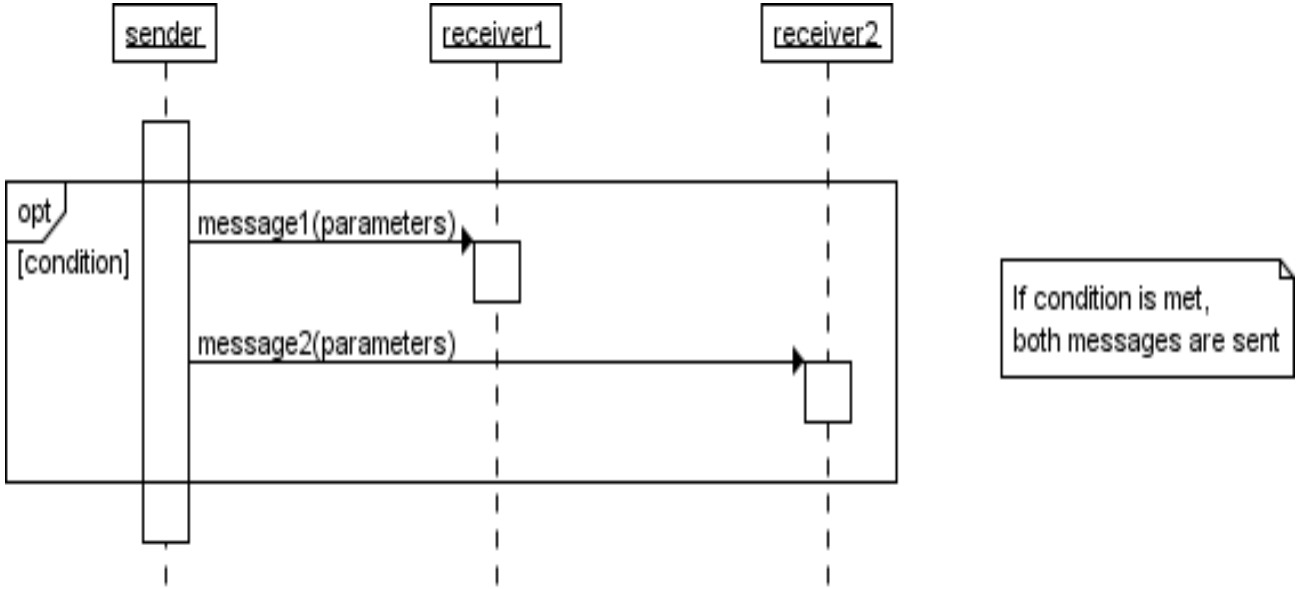


Sequence Diagrams cont.



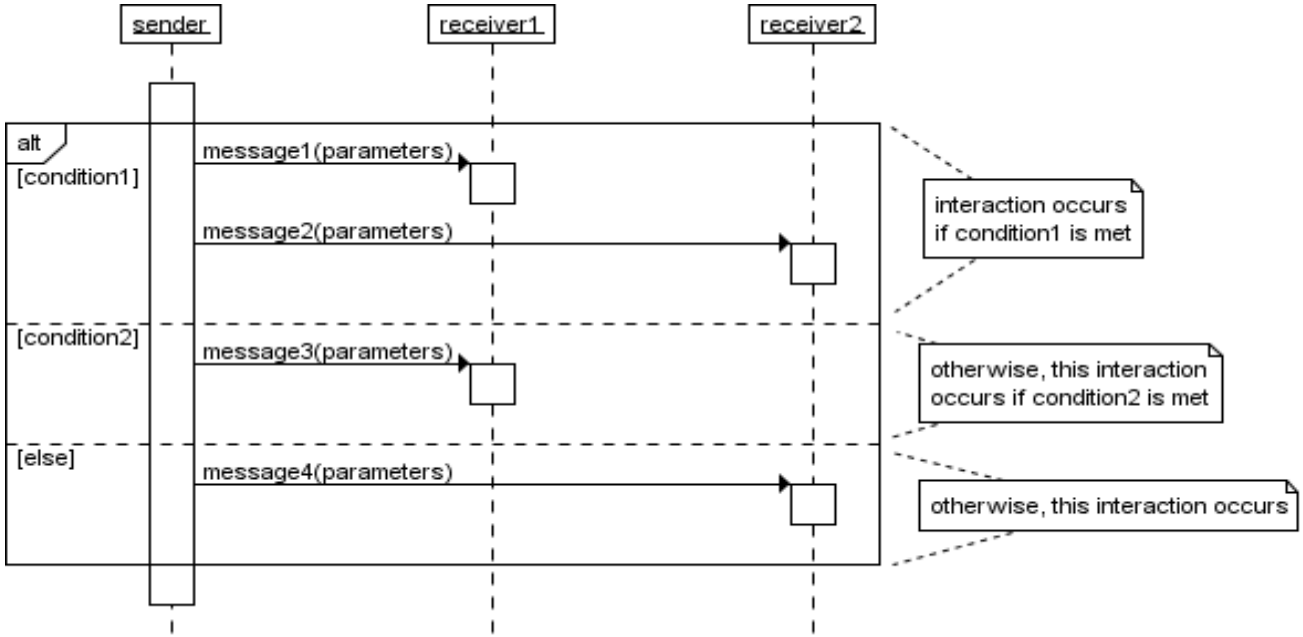
Object's **life line** and **activation bar**

Indicating selection



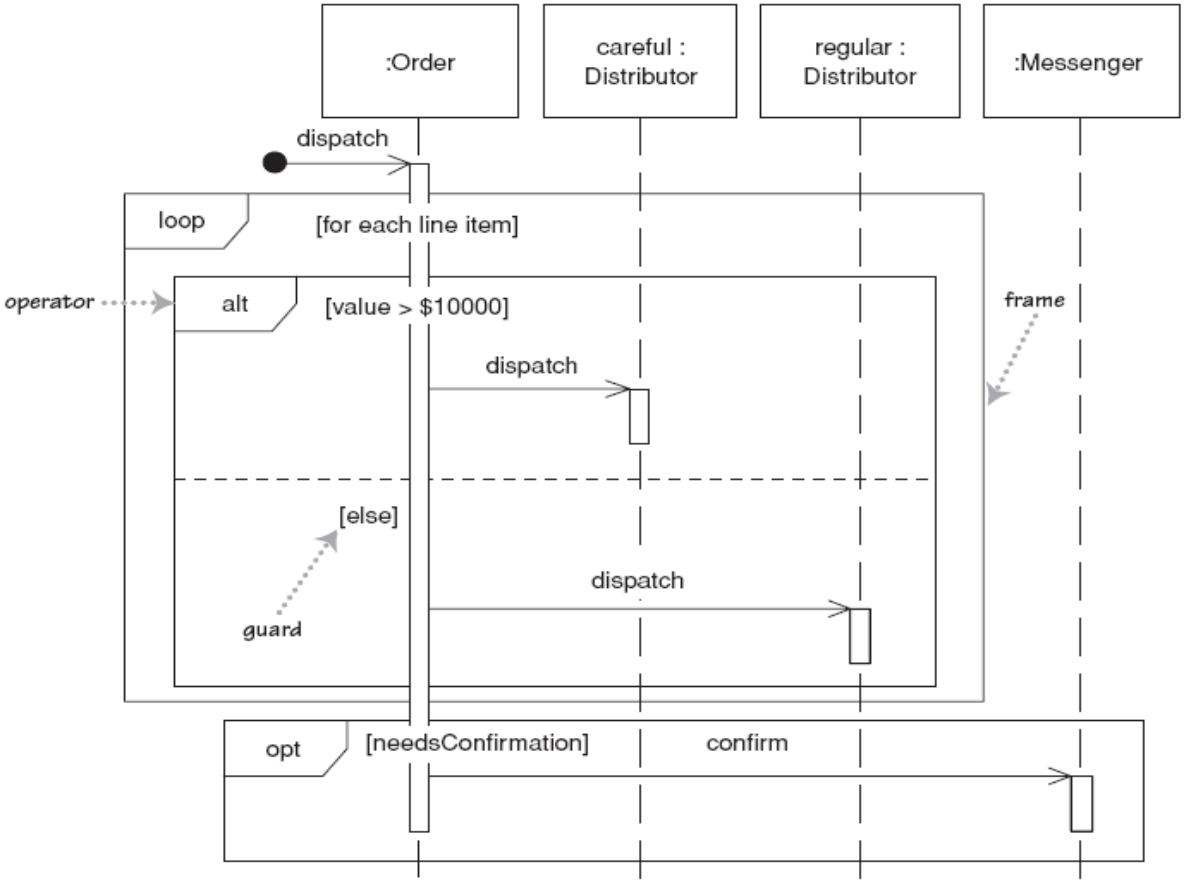
- Several messages are conditionally sent under the same guard
- The combined fragment is shown as a large rectangle with an
- 'opt' operator plus a guard

Indicating selection cont.



- Several alternative interactions, use an 'alt' combined fragment
- At most one of the operands can occur (similar to nested if-then-else and switch/case constructs)

Indicating selection and loops



Sequence Diagram-Summary

- Sequence Diagram is an interaction diagram that details how operations are carried out
  - Name classes consistently with your class diagram (same classes).
  - Include instance names when objects are referred to in messages or when several objects of the same type exist in the diagram.

Suggested reading

- Suggested reading for this lecture
  - UML Distilled, by Martin Fowler & Kendal Scott, Chapters 1-4
  - http://www.uml-diagrams.org/uml-24-diagrams.html

- **Next Lecture**
  - How to model the object's decision what to do when receiving a message using State Machine Diagrams and Activity Diagrams
  - SOLID Principles of object oriented design